

APIC / MPM 2D

Cartographie du code, opportunités d'optimisation du temps de calcul, critique ergonomique de l'interface

Résumé

Le dépôt contient un solveur 2D de dynamique des fluides et des solides par méthode des points matériels (APIC pour le fluide, MLS-MPM corotationnel pour le solide) écrit en Taichi, exécuté sur GPU, et une interface Qt pour définir des scènes, les lancer et les observer. Le cœur actif tient dans le paquet **ui/** ($\approx 1\,650$ lignes) et le module solide **Code_tuto/mpm_solid.py** ; le reste du dépôt ($\approx 1\,300$ lignes) est constitué de scripts historiques autonomes qui ont servi de tutoriel et de prototypes.

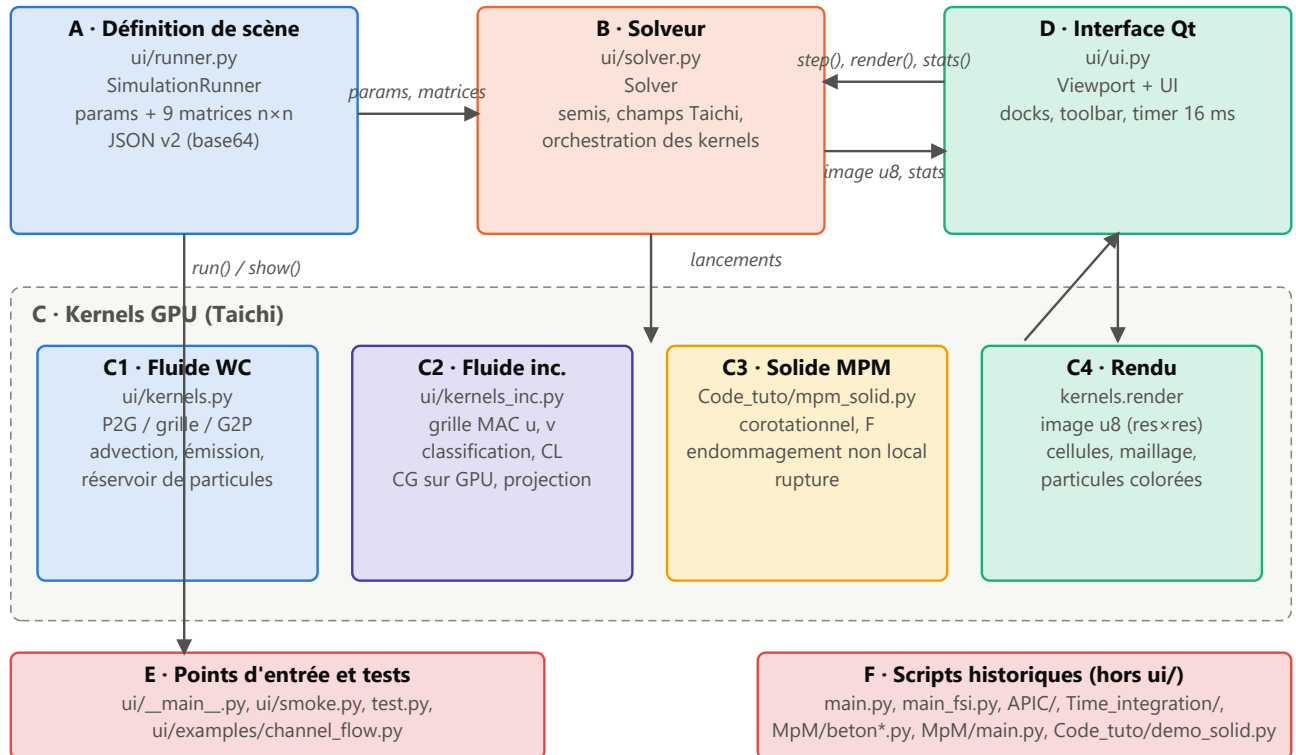
Trois constats chiffrés sur la RTX 5070 Ti (mesures de cette revue, détail en section 3) :

- **Le sous-cyclage du solide domine le couplage fluide-structure.** Sur la scène *pont_incompressible.json*, chaque pas fluide entraîne 51 sous-pas solides de 1,00 ms (8 à 9 lancements de kernels chacun) : environ deux tiers des 179 ms par image. Un seul kernel fusionné fait le même travail en 0,49 ms (prototype mesuré).
- **Le gradient conjugué converge trop lentement sans préconditionnement.** Sur *von_karman.json* (256^2 , domaine plein), la projection coûte 48 ms par sous-pas, soit plus de 85 % de l'image ; après 150 itérations le résidu relatif n'est descendu qu'à $3e-4$ et la divergence maximale résiduelle vaut 0,25 (elle devrait tendre vers 0). Un préconditionneur multigrille ramènerait le compte d'itérations sous 20 quel que soit n .
- **Chaque Reset recompile les kernels** (1,3 s avec le cache disque chaud, 4,7 s à froid) parce que les kernels sont instanciés par *template* sur des champs recréés à chaque reconstruction. C'est le premier frein à l'itération interactive sur une scène.

Côté interface, l'outil est un bon éditeur de scène mais pas encore un poste de travail de simulation : il manque la **timeline avec cache** (revenir en arrière, scruber), les **moniteurs** (résidu, divergence, énergie, dt), les **sondes** et mesures (forces sur un obstacle, profils), l'**undo**, une description de scène par **objets paramétriques** plutôt que par matrices de pixels, et la séparation du calcul et de l'affichage dans des fils distincts. La section 4 détaille ces points en s'appuyant sur les pratiques de Houdini, ParaView, Fluent, Blender et PreonLab.

1. Vue d'ensemble : les gros blocs

Le code se lit comme trois couches et deux annexes. La couche **A** décrit une scène sans rien calculer ; la couche **B** transforme cette description en champs GPU et enchaîne les kernels de la couche **C** ; la couche **D** pilote B depuis une fenêtre Qt. Les blocs E et F sont des points d'entrée et des scripts autonomes qui n'entrent pas dans la boucle interactive.

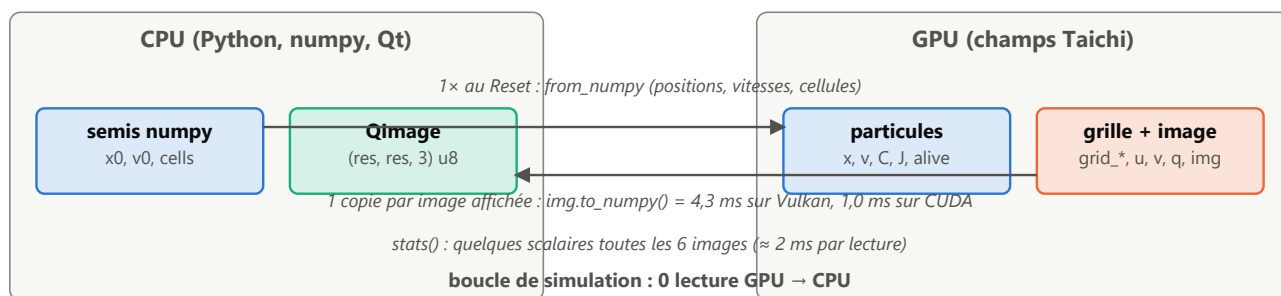


Bloc	Fichiers	Lignes	Rôle	Dépend de
A · Scène	ui/runner.py	214	Paramètres plats + 9 matrices n×n (fluid, solid, obstacle, inlet, outlet, vx0, vy0, inlet_vx, inlet_vy) ; primitives rect/circle ; JSON v2 ; resize ; run/show	numpy
B · Solveur	ui/solver.py	367	Semis des particules, allocation d'un FieldsBuilder, dt, deux boucles de sous-pas (WC et incompressible), stats, rendu	A, C
C1 · Fluide WC	ui/kernels.py	332	APIC faiblement compressible avec réservoir de particules (alive + pile de slots libres), émission, quantité colorée, rendu	Taichi
C2 · Fluide inc.	ui/kernels_inc.py	313	Grille MAC, transfert APIC sur faces, classification, CL, gradient conjugué sans lecture, projection, force de pression sur le solide	C1 (constantes)
C3 · Solide	Code_tuto/mpm_solid.py	260	MLS-MPM corotationnel, endommagement non local à vitesse limitée, rupture, couleurs, stats	Taichi
D · Interface	ui/ui.py	550	Viewport (zoom, pan, sélection), docks Paramètres et Cellules, toolbar, barre d'état, boucle QTimer, rebuild, fichiers	A, B
E · Entrées	ui/_main_.py, ui/smoke.py, ui/examples/channel_flow.py, test.py	≈ 180	Lancement, test de fumée à 5 scènes, exemple scripté, allée de von Kármán	A

Bloc	Fichiers	Lignes	Rôle	Dépend de
F · Historique	main.py, main_fsi.py, APIC/APIC.py, Time_integration/ MpM/main.py, MpM/beton.py, MpM/beton_section.py, Code_tuto/demo_solid.py	≈ 1 300	Prototypes GGUI autonomes ; béton armé (lois Desayi, acier bilinéaire) ; référence RDM	Taichi, GGUI

1.1 Chemin des données GPU ↔ CPU

Le principe de conception, respecté par le code, est qu'aucune donnée ne redescend du GPU pendant la simulation : les positions semées montent une fois, la boucle de sous-pas ne fait que des lancements, et l'image rendue est la seule copie par image affichée. Les statistiques sont lues toutes les six images et coûtent une synchronisation complète du pipeline à chaque fois.



1.2 Modes de calcul

Mode	Fluide	Pas de temps	Sous-pas / image	Couplage solide
Faiblement compressible (défaut)	APIC sur grille collocalisée, pression $p = E(1 - J)$ déposée comme contrainte dans P2G	CFL acoustique : $dt = cfl \cdot dx / \max(c_{\text{fluide}}, c_{\text{solide}}, v_{\text{entrée}})$	20 (paramètre)	Grille partagée : P2G fluide puis P2G solide additif, une seule mise à jour de grille
Incompressible	Grille MAC (u sur faces verticales, v sur faces horizontales), projection de pression par CG	CFL d'advection : $dt = cfl \cdot dx / v_{\text{max}}$, borné par $0,5 \sqrt{dx/g}$	2 à 4	Partitionné explicite : solide sous-cyclé à dt_{solid} , cellules MOVING imposent la vitesse aux faces, retour $-\text{grad } p / \rho_s$ sur les nœuds

2. Détail bloc par bloc

2.1 Bloc A — SimulationRunner (ui/runner.py)

Un objet **sans Taichi**, importable partout, qui porte la définition d'une scène : un dictionnaire de paramètres p (copie de PARAMS surchargée) et un dictionnaire de matrices m indexées $[i, j] = (x, y)$ comme les champs Taichi. Ses sous-blocs :

- **Géométrie** : n , dx , $band$ (épaisseur de la bande de paroi = $bound \cdot dx$), `centers()` renvoie les grilles X, Y des centres de cellules, `rect()` et `circle()` renvoient des masques booléens.
- **Définition** : `set_fluid` / `set_solid` / `set_obstacle` sont exclusifs (le dernier appel gagne), `set_inlet` / `set_outlet` exclusifs entre eux, `set_velocity` écrit $vx0/vy0$, `clear` efface tout sur le masque. `resize()` rééchantillonne au plus proche voisin quand n change dans l'interface.
- **Fichiers** : `to_dict` / `from_dict` sérialisent les matrices non nulles en base64 (packbits pour les booléens, float32 brut sinon) ; `save` / `load` ; `equals` sert au test aller-retour.
- **Exécution** : `solver()` instancie un Solver (import paresseux de Taichi), `run(frames, callback)` calcule sans fenêtre, `show()` ouvre l'interface Qt ; `demo()` reconstruit la scène de `main_fsi.py` (bassin, bloc d'eau, tablier sur deux piliers) en $n = 250$.

Paramètres : 24 clés. Sept sont **structurelles** (n , $bound$, ppc , $capacity$, $seed$, res , $incompressible$) et exigent un nouveau solveur ; les autres (gravité, CFL, sous-pas, matériaux, endommagement, cg_iters , $free_surface$, $color_mode$) s'appliquent à chaud via `Solver.set_params`.

2.2 Bloc B — Solver (ui/solver.py)

Le constructeur fait tout le travail CPU en une fois, puis la boucle ne touche plus au CPU.

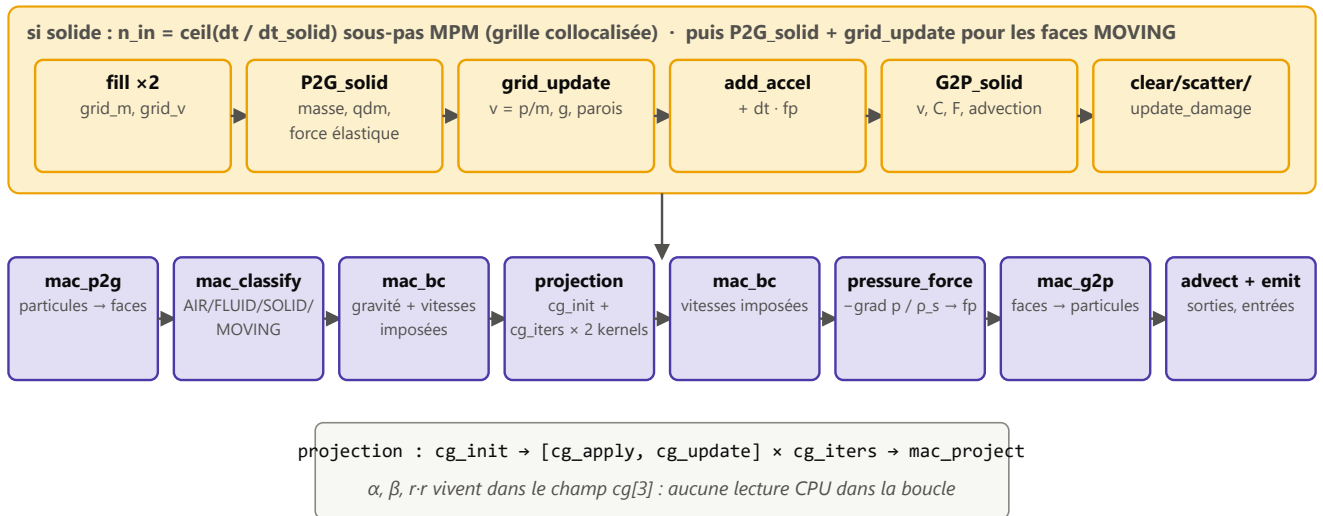
Sous-bloc	Ce qu'il fait	Points d'attention
<code>ensure_taichi()</code>	<code>ti.init</code> une seule fois par processus : APIC_UI_ARCH sinon Vulkan → CUDA → CPU	Une seule arch par processus ; impossible de changer sans redémarrer
Semis	<code>_fluid_points</code> : ppc^2 points aléatoires par cellule (graine) ; <code>_solid_points</code> : réseau régulier $ppc \times ppc$; vitesse initiale lue dans $vx0/vy0$; positions bornées à $[band, 1 - band]$	Le solide est semé en réseau, le fluide au hasard ; pas de jitter possible sur le solide
Grille de bits	<code>cells = FLUID0 SOLID0 OBSTACLE INLET OUTLET</code> , entrées/sorties tronquées à la zone utilisable (hors bande) ; <code>bc_v</code> porte (vx, vy) imposées	Une entrée dans la bande est silencieusement ignorée
Capacités	<code>capacity = max(n_fluid_init, p.capacity ou $ppc^2 \cdot n^2$)</code> si entrée ; sinon <code>n_fluid_init</code> ; ≥ 1	Sans <code>capacity</code> explicite, $4 \cdot n^2$ slots : toutes les boucles particules parcourent la capacité, pas les vivantes
FieldsBuilder	Tous les champs (particules fluide et solide, grille, image, MAC + CG si incompressible) dans un seul arbre libéré par <code>release()</code>	Les champs d'un même <code>place()</code> sont entrelacés (AoS) : <code>sc_f</code> et <code>free_stack</code> partagent la ligne de cache de <code>x_f, v_f, C_f</code>
<code>set_params()</code>	Recalcule dx , masses, Lamé, vitesses d'ondes, dt , dt_solid , τ_D (borné à $2 \cdot dt$)	Lit deux fois <code>bc_v.to_numpy()</code> alors que <code>_inlet_speed</code> est déjà calculé
<code>reset()</code>	<code>from_numpy</code> des positions/vitesses semées, <code>init_pool</code> (pile de slots libres), <code>init_solid_state</code>	Ne remet pas <code>cell_count</code> , <code>grid_*</code> : ils sont réécrits au premier sous-pas
<code>step()</code>	Choisit la boucle selon incompressible, <code>ti.sync()</code> pour mesurer <code>last_step_ms</code> , avance t	Le <code>ti.sync()</code> force l'attente du GPU dans le fil Qt
<code>stats()</code>	<code>count_alive</code> , <code>solid_stats</code> , <code>cg_rr</code> , <code>div_max</code> , <code>scalar_absmax</code> (échelle de couleur lissée 0,7/0,3)	4 à 5 lectures GPU→CPU par appel (≈ 2 ms chacune)
<code>render()</code>	<code>solid_colors</code> + <code>fluid_scalar_*</code> + kernel render dans <code>img</code> ($res \times res$ u8) puis <code>img.to_numpy()</code>	Mesuré 5,3 ms au total à 700^2 dont 4,3 ms de copie

2.2.1 Sous-pas faiblement compressible (`_substep`)



Huit à neuf lancements par sous-pas, vingt sous-pas par image : ≈ 170 lancements par image. Mesuré sur la scène demo ($n = 250, 71\,968$ particules fluides, $10\,560$ solides) : 28 ms par image soit $1,40$ ms par sous-pas.

2.2.2 Sous-pas incompressible (_substep_incompressible)



Chaque pas fluide contient donc : $n_{in} \times (8 \text{ à } 9)$ lancements pour le solide, 1 P2G_solid + grid_update supplémentaires pour les faces MOVING, 7 kernels fluides, et $1 + 2 \times cg_iters$ lancements pour la projection. Avec $cg_iters = 150$ et $n_{in} = 51$ (pont), c'est ≈ 770 lancements par pas fluide, $\approx 1\,540$ par image.

2.3 Bloc C1 — kernels du fluide faiblement compressible (ui/kernels.py)

Kernel	Boucle	Lit	Écrit	Rôle
init_pool	particules	—	alive, x, C, J, free_stack, free_top	n_{init} vivantes, les autres garées en $(-1, -1)$ et empilées
P2G_fluid	grille puis particules	x, v, C, J, alive	grid_m, grid_v (atomiques)	Remise à zéro + dépôt B-spline quadratique 3×3 , $\text{stress} = -dt \, E \, V \, (J - 1) / dx^2$
grid_update	grille	grid_m, cells, bc_v	grid_v	qdm → vitesse, gravité, parois glissantes sur bound cellules, obstacle $v = 0$, entrée $v = bc_v$
G2P_fluid	particules	grid_v, cells, bc_v	v, C, J	APIC : $C = \sum w \cdot \text{outer}(v, d) \cdot 4/dx^2$; $J^* = 1 + dt \, tr \, C$; entrée : v imposée, $C = 0$
advect_fluid	grille puis particules	v, cells	x, alive, cell_count, free_stack, free_top	$x += dt \, v$, clamp bande ; sortie : mort + push ; entrée : comptage
emit	grille	cells, cell_count, bc_v	x, v, C, J, alive, free_top	Complète chaque cellule d'entrée à target particules en dépilant des slots
fluid_scalar_wc / _inc	particules	v, J ou u, v, q, grid_v	sc_f	Quantité colorée : $ v $, vx, vy, pression, vorticité
scalar_absmax, count_alive	particules	sc_f, alive	retour scalaire	Réductions par atomiques (lecture CPU)
render	image, particules fluides, particules solides	cells, x_f, sc_f, x_s, col_s	img u8	Fond (cellules, teintes, maillage) puis splat carré de rayon r_{px} par particule

Le réservoir de particules est l'idée clé du bloc : la capacité est fixe, la vie d'une particule est un drapeau, et une pile de slots libres sur GPU (free_stack, free_top avec atomic_add / atomic_sub) relie les sorties aux entrées sans jamais compacter ni lire le CPU.

2.4 Bloc C2 — kernels incompressibles (ui/kernels_inc.py)

Kernel	Boucle	Rôle
mac_p2g	faces u, faces v, particules, faces u, faces v	Remise à zéro, dépôt APIC séparé sur les deux grilles décalées (stencil décalé de 1/2 cellule), normalisation par les poids mu, mv
mac_classify	cellules, particules solides, particules fluides	SOLID (paroi, obstacle, entrée) ; MOVING si une particule solide s'y trouve ; FLUID si une particule fluide (ou domaine plein) ; AIR sinon et sorties
mac_bc	faces v, faces u, faces v	Gravité sur v (optionnelle), puis vitesse imposée sur toute face touchant SOLID/MOVING : 0, bc_v, ou moyenne massique des deux nœuds solides
cg_init	cellules ×2	$rhs = -dx \cdot div$; $r = b - A q$ (démarrage à chaud sur la q précédente) ; $p = r$; $cg[0] = r \cdot r$
cg_apply	cellules	$Ap = A p$ sur FLUID (laplacien 5 points, voisin AIR : $q = 0$, voisin SOLID exclu) ; $cg[1] = p \cdot Ap$
cg_update	cellules ×2	$\alpha = cg[0]/cg[1]$; $q += \alpha p$; $r -= \alpha Ap$; $cg[2] = r \cdot r$; $\beta = r + \beta p$; $cg[0] = cg[2]$
mac_project	faces u, faces v	$u -= (q_e - q_w)/dx$ entre deux cellules non solides dont l'une est fluide
pressure_force	grille collocalisée	$fp = coef \cdot grad q$ (moyenne des 4 cellules autour du nœud, $q = 0$ hors FLUID) sur les nœuds massiques du solide
add_accel	grille	$grid_v += dt \cdot fp$ (appliqué à chaque sous-pas solide)
mac_g2p	particules	v et C reconstruits face par face en ignorant les faces sans poids ($\mu = 0$) ; sinon la particule garde sa vitesse
max_speed, divergence_max, cg_residual	particules / cellules	Diagnostics et CFL (lectures CPU)

Le système résolu est $A q = -dx \cdot div(u)$ avec $q = dt p / p$ et A le laplacien positif ; les réductions $r \cdot r$ et $p \cdot Ap$ sont des accumulations sur un scalaire de kernel, que Taichi implémente par atomiques sur les backends GPU. Le point de départ à chaud (q précédente) est ce qui rend 150 itérations suffisantes sur les scènes à surface libre.

2.5 Bloc C3 — kernels du solide (Code_tuto/mpm_solid.py)

- **kirchhoff_stress** : $\tau = 2\mu (F - R) F^T + \lambda J (J - 1) I$, $R = U V^T$ par SVD 2×2 (modèle corotationnel de Stomakhin 2012).
- **P2G_solid** (additif, ne remet pas la grille à zéro) : raideur effective $k = \max(1 - D, k_{res})$, 1 si rompue ; affine = $(-dt V 4/dx^2) \tau + m C$.
- **G2P_solid** : v, C ; $F \leftarrow (I + dt C) F$; si rompue, valeurs singulières écrêtées dans $[0, 1 ; 1]$ (pas de traction) ; advection et clamp bande.
- **clear_eps / scatter_eps / update_damage** : l'allongement principal $\max(\sigma) - 1$ des particules saines est lissé par la grille (non local, ≈ 2 cellules), D suit $\text{clamp}((\epsilon - \epsilon_0)/(\epsilon_f - \epsilon_0))$ à vitesse limitée dt/τ_D , rupture à $D = 1$.
- **solid_colors, solid_stats, init_beam** : affichage (endommagement ou déformation), (n rompues, D max), semis en réseau.

Quatre SVD par particule et par sous-pas (P2G, G2P si rompue, scatter_eps, update_damage) ; la fonction grid_update de ce module n'est pas utilisée par ui/ (remplacée par kernels.grid_update, sans amortissement).

2.6 Bloc D — interface Qt (ui/ui.py)

Sous-bloc	Contenu	Comportement
Viewport (QWidget)	x0, y0, scale ; _buf / _qimg ; sel ; _drag ; _pan	Molette : zoom $\times 1,25^k$ autour du curseur (1 à 64) ; gauche : boîte de cellules (signal selectionChanged) ; droit/milieu : pan ; F : vue entière ; Échap ; paintEvent dessine l'image dans le carré inscrit puis le rectangle de sélection
Dock Paramètres	QTreeWidget : Domaine, Solveur, Matériaux (Fluide, Solide), État initial, Conditions aux limites	Éditeurs générés depuis PARAMS (QCheckBox / QSpinBox / QDoubleSpinBox, keyboardTracking off) ; structurel → resize/dirty, sinon set_params à chaud ; résumés recalculés à chaque modification
Dock Cellules	vx, vy ; 7 boutons (Eau, Solide, Obstacle, Entrée, Sortie, Vitesse, Effacer) ; maillage, teintes ; couleur du fluide ; échelle	Applique la fonction du runner sur le masque rectangulaire de la sélection, puis _mark_dirty (bandeau orange)

Sous-bloc	Contenu	Comportement
Toolbar / raccourcis	Play (Espace), Step (S), Reset (R), Vue entière (F), Ouvrir (Ctrl+O), Enregistrer (Ctrl+S / Ctrl+Maj+S)	Play ou Step sur une scène modifiée déclenche _rebuild
Boucle _tick	QTimer 16 ms	step() si playing ; render() et set_image à chaque tick ; stats et barre d'état toutes les 6 images
_rebuild	release() de l'ancien solveur, nouveau Solver, step(1) pour compiler, reset()	Bloquant dans le fil Qt (message dans la barre d'état + processEvents)
Fichiers / titre	_open, _save, _save_as, _update_title	Le JSON du runner ; l'état des particules n'est pas sauvegardé
APIC_UI_AUTOQUIT	autoplay + autoquit	Diagnostic sans intervention (tests)

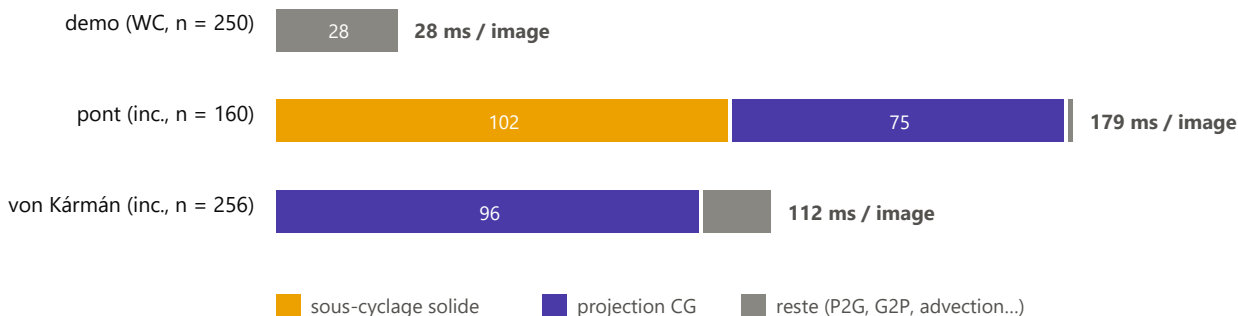
2.7 Blocs E et F — entrées, tests, scripts historiques

Fichier	Statut	Contenu
ui/__main__.py	actif	python -m ui [scène.json] : charge ou demo(), show()
ui/smoke.py	actif	5 scènes sans fenêtre : demo (stats, rendu, zoom), paramètres à chaud + JSON, entrée/sortie, barrage incompressible (div < 1e-2), poutre légère qui remonte (Archimède)
ui/examples/channel_flow.py	actif	Canal avec jet, rocher, socle, lame solide ; 150 images + stats ; --show, --save
test.py	actif	Allée de von Kármán : conduit plein, cylindre, entrée/sortie à U = 0,5, incompressible sans surface libre, n = 256
main.py + APIC/APIC.py + Time_integration/	historique	Premier solveur APIC GGUI : bloc d'eau sur un cylindre, dt CFL calculé sur GPU à chaque pas ; P2G/G2P identiques à kernels.py sans réservoir
main_fsi.py	historique	Eau + pont sur grille commune, GGUI ; c'est la scène demo() de ui/
Code_tuto/demo_solid.py, MpM/main.py	historique (doublon)	Poutre encastrée seule, charge réglable, modèles 1/2/3 ; MpM/main.py est une copie plus ancienne de mpm_solid + demo dans un seul fichier, avec E = 30 000
MpM/beton.py, MpM/beton_section.py	historique, spécifique	Poutre en béton armé (flexion 4 points, lois Desayi/Neville, acier bilinéaire, résistances lognormales, vérins à vitesse imposée, courbe P-flèche CSV) et référence RDM par analyse de section
cas.json, von_karman.json, pont_incompressible.json	données	Scènes sauvegardées (v2) ; cas.json fait 730 ko car les matrices float32 ne sont pas compressées

3. Opportunités d'optimisation du temps de calcul

3.1 Où va le temps : mesures

Mesures réalisées avec les scripts docs/bench/bench.py, bench2.py (prototypes de kernels fusionnés) et bench3.py (copies d'image, réductions) sur les trois scènes de référence, backend Vulkan sauf mention, GPU RTX 5070 Ti, Taichi 1,7.4. Les kernels sont chronométrés après échauffement avec ti.sync() ; les valeurs sont dans docs/bench/results.json et ce document se régénère avec docs/bench/make_pdf.py.



Répartition du temps par image (ms), somme des composantes mesurées séparément ; la scène demo n'a ni projection ni sous-cyclage.

Mesure	Vulkan	CUDA	Commentaire
Lancement d'un kernel minuscule	0,11 ms	0,11 ms	Surcoût pur Python + runtime ; plancher de tout kernel
Lecture d'un scalaire GPU → CPU	2,1 ms	1,2 ms	Synchronisation complète : à bannir des boucles
Une itération CG, pont (7 442 cellules fluides / 25 600)	0,34 ms	0,57 ms	2 lancements ; limité par le lancement, pas par le calcul
Une itération CG, von Kármán (≈ 65 000 cellules)	0,33 ms	0,45 ms	Le calcul commence à compter
Itération CG fusionnée en 1 kernel (prototype)	0,20 / 0,32 ms	0,35 / 0,43 ms	pont / von Kármán : gain seulement quand le lancement domine
Réduction r·r sur 65 536 cellules : atomique global / par ligne / stencil seul	0,17 / 0,11 / 0,09 ms	0,30 / 0,41 / 0,12 ms	L'accumulation atomique double le coût d'un passage de stencil
Copie de l'image 700 ² ×3 u8 (variantes : champ vectoriel, plat, i32, ndarray)	4,0 à 5,3 ms	3,0 à 4,7 ms	Plancher de synchronisation, indépendant du format
Sous-pas solide (9 lancements)	1,00 ms	1,25 ms	× 51 par pas fluide × 2 pas par image
Sous-pas solide fusionné en 1 kernel (prototype)	0,49 ms	0,48 ms	Mêmes calculs, un seul lancement
Image demo : ms par image (20 sous-pas)	28 ms	29 ms	≈ 170 lancements
Image pont	179 ms	206 ms	≈ 1 540 lancements
Image von Kármán	112 ms	127 ms	≈ 320 lancements, dominés par le CG
render() complet à 700 ² , vue entière	5,3 ms	1,8 ms	kernel 0,5 ms + copie 4,3 ms + solid_colors
render() au zoom 64×	14,7 ms	4,2 ms	Le splat carré tourne pour les particules hors champ
Reconstruction du solveur (Reset structurel)	1,3 s	2,3 s	Cache disque chaud ; 4,7 s à froid

Convergence du gradient conjugué (résidu relatif r·r / r0·r0, démarrage à chaud) :

Scène	Cellules fluides	Itérations pour 1e-2	pour 1e-4	pour 1e-6	Résidu relatif à 150 it.
pont (surface libre)	7 442	—	123	> 150	3,7e-6
von Kármán (domaine plein)	60 480	27	> 400	> 400	3,3e-4

Lecture : le CG non préconditionné a besoin d'un nombre d'itérations proportionnel à la taille du domaine ($\approx n$ pour un laplacien 2D). Sur le domaine plein 256^2 , 150 itérations laissent un champ de pression loin de convergé ; la divergence résiduelle est absorbée image après image par le démarrage à chaud, ce qui fonctionne visuellement mais fausse les pressions et les forces mesurées.

3.2 Pistes classées par gain / effort

#	Piste	Constat et proposition	Gain estimé	Effort	Risque / remarque
1	Fusionner le sous-pas solide en un kernel	Un kernel à 5 boucles de premier niveau (clear, P2G, grille, G2P + scatter, damage) : mesuré 1,00 $\rightarrow$ 0,49 ms. Sur le pont, $\approx$ 102 ms par image deviennent $\approx$ 50 ms.	$\times$ 2,0 sur le solide, image pont $\approx$ 127 ms	faible	Aucun changement de résultat ; garder les kernels séparés pour les tests
2	Préconditionner la projection (MGPCG)	Multigrille géométrique en préconditionneur du CG, comme mgpcg_advanced.py des exemples Taichi : 3 à 4 niveaux, lisseur Jacobi amorti ou rouge-noir, un V-cycle par itération. Le compte d'itérations tombe à 10–20 quel que soit n . Chaque V-cycle peut tenir dans un seul kernel (boucles par niveau déroulées avec ti.static).	$\times$ 5 à 8 sur la projection ; von Kármán $\approx$ 100 $\rightarrow$ 20 ms	moyen–élevé	Le cas surface libre + cellules MOVING demande de restreindre correctement le masque ; valider avec div_max
3	Fusionner l'itération CG en un kernel	cg_apply + cg_update en un kernel à 3 boucles : 0,34 $\rightarrow$ 0,20 ms sur le pont (7 442 inconnues, lancement dominant) mais 0,33 $\rightarrow$ 0,32 ms sur von Kármán (60 480 inconnues, calcul dominant). Compatible avec la piste 2.	$\times$ 1,7 sur la projection en surface libre ; rien sur un domaine plein	faible	—
4	Réductions sans atomique global	r-r et p-Ap accumulent 60 000 atomiques flottants sur une seule adresse par kernel : sur 65 536 cellules, un passage de stencil coûte 0,09 ms seul et 0,17 ms avec l'accumulation. Une réduction en deux étages ($\approx$ 1 024 sommes partielles par autant de threads, puis une somme finale) rend l'accumulation presque gratuite ; la variante « une ligne par thread » testée gagne sur Vulkan (0,11 ms) mais perd sur CUDA (trop peu de threads). Même idée pour scalar_absmax, count_alive, max_speed, divergence_max.	$\times$ 1,5 à 2 sur chaque kernel CG à 256^2	faible	Mesurer les deux backends ; garder la version atomique en référence
5	Réduire le sous-cyclage solide	$n_{in} = 51$ vient de $dt_solid = cfl \cdot dx / c_s$ avec $c_s \approx 45$ m/s. Options : mettre à jour l'endommagement une fois par pas fluide au lieu de 51 (3 kernels sur 9), fusionner les deux fill dans P2G_solid, et surtout accepter un dt fluide plus petit quand le solide domine (substeps = 4 réduit n_{in} à 13 pour le même coût total mais un couplage plus précis).	$\times$ 1,3 à 1,5 en plus de la piste 1	faible	L'endommagement à vitesse limitée dépend de dt : garder dt/τ_D cohérent
6	Éviter la recompilation au Reset	Les kernels prennent les champs en ti.template() : chaque FieldsBuilder est une nouvelle instanciation (1,3 s). Passer aux arguments ti.types.ndarray() (compilés une fois par type et rang) ou allouer une seule fois à la capacité max et ne réinitialiser que les contenus.	Reset de 1,3 s $\rightarrow$ $\approx$ 50 ms	moyen	Les ndarrays supportent les atomiques et ti.grouped ; l'arbre FieldsBuilder disparaît

#	Piste	Constat et proposition	Gain estimé	Effort	Risque / remarque
7	Rendu : rejeter les particules hors champ, ne rendre que si nécessaire, accepter le plancher de copie	Dans render, tester le rectangle [col - r, col + r] contre l'image avant la boucle de splat (zoom 64x : 15 ms → ≈ 5 ms). Ne pas appeler render() quand la simulation est en pause et que ni la vue ni les options n'ont changé. La copie de l'image coûte 4,3 ms sur Vulkan quel que soit le format (champ vectoriel, plat, i32, ndarray : toutes les variantes mesurées entre 4 et 5 ms) : c'est un plancher de synchronisation, pas un débit. Le seul moyen de l'éviter est l'affichage zéro copie (texture Vulkan partagée avec un QRhiWidget), un chantier à part ; la piste 12 la masque en la sortant du fil Qt.	Zoom fort × 3 ; pause : 5 ms par tick économisés	très faible	—
8	Choisir le backend par scène	APIC_UI_ARCH=cuda ne change pas le code. Mesuré ici : CUDA est plus lent sur les kernels (pont 206 contre 179 ms, von Kármán 127 contre 112 ms) et plus rapide sur les lectures et la copie d'image (1,0 contre 4,3 ms). Vulkan reste le bon défaut pour la simulation ; CUDA sert au débogage des accès mémoire.	0 à 15 % selon la scène	nul	Une seule arch par processus
9	Compacter les particules vivantes	Avec une entrée, capacity = 4 n ² (340 787 slots pour 248 079 vivantes sur von Kármán) : toutes les boucles particules parcourent la capacité. Un compactage périodique (préfixe-somme Taichi, ti.algorithms.PrefixSumExecutor) ou une capacité dimensionnée réduit les boucles au nécessaire.	10 à 25 % sur P2G/G2P	moyen	L'ordre des particules change : sans effet physique
10	Liste compacte des cellules fluides pour le CG	Sur une surface libre (pont : 29 % de cellules fluides), les boucles CG parcourent n ² cellules et testent ctype. Construire une liste d'indices FLUID dans mac_classify et itérer dessus divise le travail par 3 ; sans effet sur un domaine plein.	× 2 à 3 sur le CG en surface libre	faible	—
11	Regrouper les lectures de stats()	count_alive, solid_stats, cg_residual, divergence_max, scalar_absmax font 4 à 5 synchronisations (≈ 2 ms chacune). Un kernel unique qui écrit un champ de 8 flottants, lu en une fois, ramène cela à une lecture.	≈ 8 ms toutes les 6 images	très faible	—
12	Séparer calcul et affichage (fil de calcul)	Le ti.sync() de step() et le render() tournent dans le fil Qt : une image de 180 ms gèle l'interface. Un QThread dédié à Taichi (tous les appels Taichi depuis ce fil) avec une file d'ordres, et le fil Qt qui ne fait que recevoir l'image u8, garde l'interface fluide et permet de rendre à 60 Hz pendant qu'un pas de 200 ms se calcule.	Réactivité, pas de temps CPU	moyen	Aucun appel Taichi depuis un slot Qt

3.3 Ordre recommandé

- **Semaine 1** : pistes 1, 3, 4, 7, 11 (fusions, réductions, rendu) — quelques heures chacune, gains mesurables immédiatement, aucun changement de physique. Le pont passe d'environ 179 ms à ≈ 96 ms par image avec les seules pistes 1 et 3.
- **Semaine 2** : piste 6 (ndarray) — c'est la plus rentable pour le confort d'itération, et elle simplifie la piste 12.
- **Ensuite** : piste 2 (MGPCG), qui change la classe de complexité de la projection et rend les résolutions 512² raisonnables ; puis 5, 9, 10 selon les scènes réellement utilisées.

- **À ne pas faire** : réintroduire une lecture CPU dans la boucle CG pour un arrêt anticipé — la lecture coûte l'équivalent de 5 à 8 itérations, et le compte fixe reste prévisible. Une alternative sans lecture : adapter `cg_iters` au résidu lu dans `stats()` toutes les 6 images.

Ce qui est déjà bien fait et à conserver : zéro copie dans la boucle, réservoir de particules sans compactage, α et β sur GPU, rendu de la vue zoomée sur GPU en u8, fills et kernels sans globales (arguments explicites), bande de paroi qui protège le stencil.

4. Critique de l'interface et propositions

4.1 Ce qui fonctionne

- Le viewport est correct : zoom autour du curseur, pan, sélection de cellules, maillage à partir de 6 px, coordonnées au survol.
- La séparation runner / solver / ui est saine : la scène est un objet sérialisable, l'interface n'a pas d'état caché, les scripts et l'interface partagent le même chemin.
- Le mécanisme dirty + bandeau orange + Reset est explicite ; les paramètres à chaud contre structurels sont distingués.
- Les raccourcis essentiels existent (Espace, S, R, F, Ctrl+O/S) et les tests de fumée couvrent le pipeline.

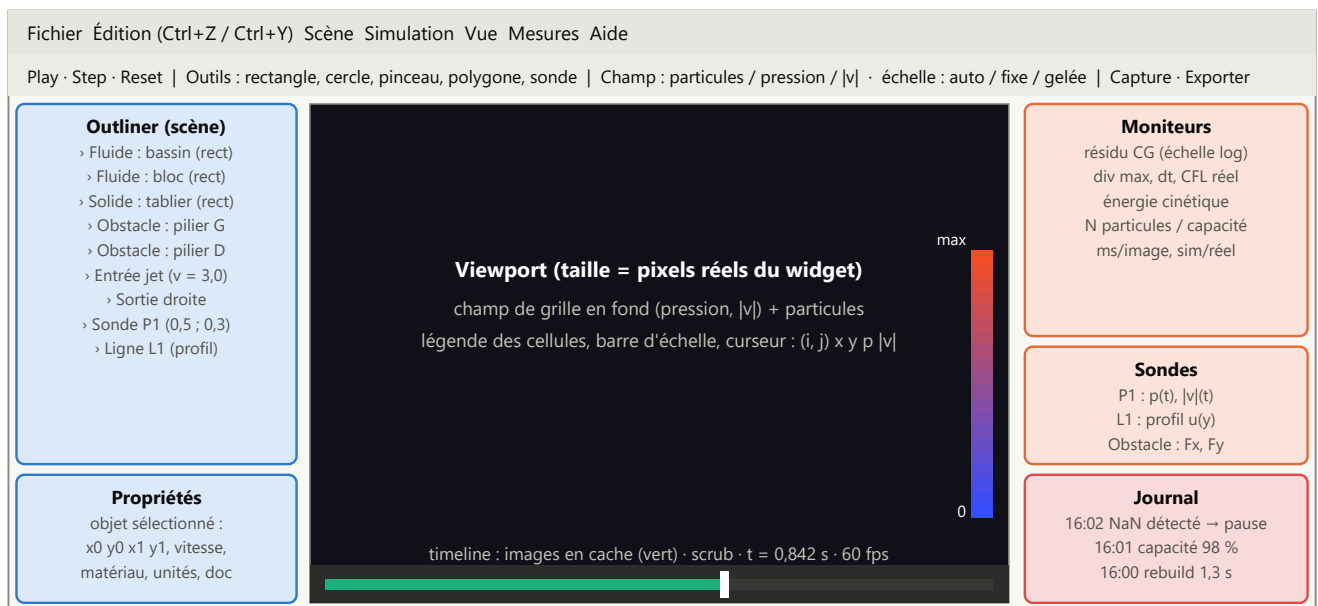
4.2 Points faibles, par ordre d'impact sur un travail de simulation

Point	Constat	Ce que font les pros
Pas de retour en arrière	Play, Pause, Step, Reset : impossible de revoir l'image précédente, de scruber, ni de comparer deux instants. Un phénomène vu trop tard (rupture, instabilité) oblige à tout relancer.	Houdini, Blender : cache de frames + timeline scrutable ; PreonLab : lecture indépendante du calcul
Aucun moniteur	<code>cg_rr</code> et <code>div_max</code> sont calculés mais jamais affichés ; pas de courbe de résidu, d'énergie, de dt, de nombre de particules dans le temps. On ne sait pas si la projection converge ni si le pas est stable.	Fluent, OpenFOAM (foamMonitor), COMSOL : fenêtre de résidus par défaut
Aucune mesure	Pas de sonde ponctuelle, de profil sur une ligne, de force sur un obstacle, d'intégrale sur une zone. Sans cela on ne peut valider (Strouhal, Cd, flèche d'une poutre).	ParaView : Probe, Plot Over Line ; Fluent : report definitions
Interface gelée pendant le calcul	<code>step()</code> bloque le fil Qt (<code>ti.sync</code>) : à 180 ms par image, les spinbox et la molette répondent par à-coups, et un rebuild de 1,3 s fige tout.	Toutes les applications pro calculent dans un processus ou un fil séparé
Scène décrite en pixels	Les objets sont des matrices booléennes : impossible de déplacer un rectangle, changer un rayon, éditer une vitesse d'entrée après coup sans tout redessiner ; changer n rééchantillonne au plus proche voisin.	Outliner + objets paramétriques (Blender, Houdini, tout CAO)
Pas d'undo	Une erreur de sélection sur « Effacer » est irréversible.	QUndoStack est standard dans Qt
Champs visibles seulement via les particules	Pas de rendu de la pression ou de la vitesse sur la grille en fond, pas de vecteurs, de lignes de courant, ni de barre de couleur dans le viewport ; l'échelle auto (lissée) change en permanence.	ParaView : colorbar, range auto/fixe/gelé ; glyphs, streamlines
Outils de dessin pauvres	Seule la boîte rectangulaire ; pas de cercle, polygone, pinceau, ligne, ni import d'un masque depuis une image PNG.	Éditeurs 2D ; import d'images pour la géométrie
Silences dangereux	Une entrée dans la bande de paroi est ignorée sans message ; la capacité pleine arrête l'émission sans avertir ; un NaN se propage sans détection ; un CFL non respecté n'est pas signalé.	Journal + avertissements bloquants
Rendu à résolution fixe 700 ²	L'image est étirée dans le carré inscrit : floue sur un grand écran ou en HiDPI, et l'espace latéral d'une fenêtre large est perdu.	Rendre à la taille réelle du widget × devicePixelRatio

Point	Constat	Ce que font les pros
Paramètres sans contexte	Pas d'unité, de plage, de tooltip, ni de valeurs dérivées (c_s , dt , Mach, nombre de particules, mémoire) ; le suffixe « * » est discret.	COMSOL/Fluent affichent les dérivées ; tooltips partout
Pas d'export	Ni capture, ni séquence d'images/vidéo, ni CSV, ni checkpoint de l'état des particules pour reprendre.	Export image/vidéo, CSV, VTK pour ParaView

4.3 Proposition d'organisation

Maquette d'un poste de travail de simulation, en gardant l'architecture actuelle (runner / solver / ui) : un outliner d'objets à gauche, le viewport au centre, moniteurs, sondes et journal à droite.



4.4 Les changements, du plus structurant au plus simple

Fil de calcul et boucle d'affichage découplés

Un QThread « SimWorker » possède le solveur et exécute tous les appels Taichi ; il reçoit des ordres (step, reset, set_params, render(vue)) par une file et émet des signaux (image prête, stats). Le fil Qt ne fait que peindre la dernière image reçue. Conséquences : l'interface reste fluide pendant un rebuild, le rendu peut tourner à 60 Hz sur une vue qui bouge même si le pas fait 200 ms, et l'on peut afficher un vrai rapport temps simulé / temps réel avec une cible (« temps réel », « au plus vite », « N pas par image »).

Timeline avec cache de frames

Conserver en mémoire GPU (ou CPU si l'on accepte une copie au moment de l'enregistrement, hors boucle) les N dernières images rendues, ou mieux l'état complet (x, v, C, J, alive, F, D) toutes les k images sous forme de checkpoints. Une barre de temps montre les images en cache, un glisser scrube, Maj+S revient d'une image, et « reprendre ici » relance depuis un checkpoint. Le coût mémoire d'un checkpoint est celui des champs (≈ 12 flottants par particule fluide : 250 000 particules ≈ 12 Mo) ; 100 checkpoints tiennent dans la carte.

Moniteurs et sondes

- Un dock « Moniteurs » avec pyqtgraph (léger, rapide) : résidu CG en échelle log, div_max, dt, CFL réel ($v_{\max} dt / dx$), énergie cinétique, nombre de particules / capacité, ms par image. Les données proviennent d'un seul kernel de stats (piste 11) lu toutes les k images.
- Sondes : un clic avec l'outil sonde crée un objet « Sonde » dans l'outliner (cellule i, j) ; sa série $p(t)$, $|v|(t)$ s'affiche. Une « Ligne » donne un profil $u(y)$ dans le canal. Un « Obstacle » ou un « Solide » expose F_x , F_y intégrés à partir de q sur ses faces (le kernel pressure_force fait déjà l'essentiel).

- Export CSV de toutes les séries en un clic, pour comparer à une référence (comme MpM/beton_section.py le fait déjà pour le béton).

Scène par objets

Remplacer la description par matrices par une liste d'objets (rect, circle, polygon, image importée) portant leur type (fluide, solide, obstacle, entrée, sortie, vitesse) et leurs paramètres. Les matrices actuelles deviennent le résultat d'une rasterisation faite par le runner au moment du build : le format JSON reste compatible (on peut stocker les deux), les scripts existants continuent de marcher, et l'interface gagne : déplacer, redimensionner, dupliquer, désactiver un objet, changer n sans perte. Un calque « peinture » (matrice libre) garde les cas particuliers.

Édition

- QUndoStack sur toutes les opérations de scène et de paramètres ; Ctrl+Z / Ctrl+Y.
- Outils : rectangle, cercle, polygone, pinceau avec rayon, ligne (pour une lame fine), pipette ; import PNG (blanc = fluide, etc.) à la résolution de la grille.
- Presets : « Nouveau depuis un modèle » (barrage, canal, von Kármán, pont, poutre) ; la scène demo devient un fichier.

Visualisation

- Champ de grille en fond (pression, $|v|$, vorticité, type de cellule) avec une barre de couleur dans le viewport, plage auto / fixe / gelée, et un mode « $\pm$ max symétrique » pour les quantités signées.
- Vecteurs de vitesse sur la grille (un sur k), lignes de courant optionnelles, particules coloriées comme aujourd'hui.
- Légende des teintes de cellules dans le viewport, barre d'échelle en unités du domaine, position du curseur avec les valeurs de champ sous le curseur.
- Rendu à la taille réelle du widget (res = pixels du widget $\times$ devicePixelRatio) ; res cesse d'être un paramètre de scène.

Robustesse et confort

- Journal (dock) avec horodatage : avertissements pour entrée dans la bande, capacité atteinte, dt tronqué, NaN détecté (pause automatique), temps de rebuild.
- Tooltips sur chaque paramètre : signification, unité, plage, effet ; panneau « Dérivées » : c_fluide, c_solide, dt, sous-pas solides, particules, mémoire.
- Fichier récent, autosave de la scène, checkpoint de l'état (npz) séparé de la scène (json).
- Export : capture PNG, séquence d'images / MP4 (ffmpeg), CSV des moniteurs, VTK des particules pour ParaView.
- Paramétrique : file d'exécution sans fenêtre (balayage d'un paramètre, N graines) avec un tableau de résultats — le runner sait déjà tout faire, il manque l'interface.

4.5 Feuille de route indicative

Phase	Contenu	Dépend de	Effet
1	Fil de calcul, kernel de stats unique, moniteurs pyqtgraph, journal + détection de NaN, colorbar et champ de grille en fond, rendu à la taille du widget	pistes 11 et 12 de la section 3	Interface fluide et instrumentée
2	Timeline + cache de checkpoints, Maj+S, capture et export image/vidéo/CSV	phase 1	Observation et communication des résultats
3	Scène par objets + outliner + undo + outils de dessin + presets	—	Itération rapide sur les cas
4	Sondes, lignes, forces sur obstacle / solide ; balayage paramétrique sans fenêtre	phases 1 et 3	Validation quantitative

Annexe — inventaire complet des fichiers

Fichier	Lignes	Statut
ui/ui.py	550	actif
ui/solver.py	367	actif
ui/kernels.py	332	actif
ui/kernels_inc.py	313	actif
ui/runner.py	214	actif
ui/smoke.py	105	actif (test)
ui/examples/channel_flow.py	45	actif (exemple)
ui/__main__.py	14	actif
ui/README.md	142	documentation
Code_tuto/mpm_solid.py	260	actif (solide)
Code_tuto/demo_solid.py	169	historique
MpM/beton.py	538	spécifique, autonome
MpM/beton_section.py	125	spécifique, autonome
MpM/main.py	348	historique (doubleton de mpm_solid + demo)
main_fsi.py	189	historique
main.py	138	historique
APIC/APIC.py	108	historique
Time_integration/time_integration.py	26	historique
test.py	31	actif (von Kármán)
cas.json / von_karman.json / pont_incompressible.json	—	scènes